

Libraries, Types in Seq. circuits

Lecture 5



Electronic and Communication Engineering Department
ECE 307: Introductory to Digital Systems Lab.
Dr. Ammar Jallawi Mahmood
Ammar.karkar@uokufa.edu.iq

Libraries

- *Libraries* provide an alternative to explicit component declaration
 - Standard Library (**STD**) – Implicit
 - Working Library (**work**) – Implicit
 - Resource Libraries
 - IEEE developed libraries
 - Intel FPGA component libraries
 - Any library of design units that are referenced in a design
- Libraries are structured in one or more *packages* that contain the declaration of types, functions and components
- Components within a declared package *do not need to be re-declared within the architecture.*
- User-defined packages are common (component reuse)

```
library ieee;  
use ieee.std_logic_1164.all;
```

Referencing libraries and packages

- *Syntax:*

```
library <name>;  
use <name>.<package_name>.all;
```

- **library** clause:

- *Defines the library name that can be reference*
- *Is a symbolic name to a path/directory*

- **use** clause

- *Specifies the package and object in the library that you have specified in the library clause*
- *All packages used must be compiled*

The WORK library

```
library work; -- library is where we save our design  
use work.all; -- (.vhd file & compiler & simulator etc files)
```

- The fact that the **work** library is implicit in all designs, provides an alternative way to instantiate components within an entity
- This notation is called **direct instantiation** or **entity instantiation**, to differentiate from the **component instantiation** described before
- It relies on the fact that components declared in a library do not need to be re-declared in the architecture

Libraries - IEEE

- The **ieee** library contains several packages:

- **std_logic_1164**
- std_logic_textio
- std_logic_arith
- std_logic_signed
- std_logic_unsigned

Packages that contain types and functions for circuit design, synthesis, and simulation

- numeric_bit
- **numeric_std**

Packages designed to replace STD_LOGIC for synthesis of arithmetic functions

- math_real
- math_complex

Non-synthesizable (mostly) functions on real and complex numbers



Packages – std_logic_1164

- This package contains the definition of *types*:
`std_ulogic` `std_ulogic_vector`
`std_logic` `std_logic_vector`
- For these types, it (re-)defines the *logic functions*:
`and` `nand` `or` `xor` `xnor` `not`

```
function "and" ( L : STD_ULOGIC; R : STD_ULOGIC) return STD_ULOGIC;  
function "and" ( L : STD_LOGIC; R : STD_LOGIC) return STD_LOGIC;  
function "and" ( L : STD_ULOGIC_VECTOR; R : STD_ULOGIC_VECTOR)  
    return STD_ULOGIC_VECTOR;  
function "and" ( L : STD_LOGIC_VECTOR; R : STD_LOGIC_VECTOR)  
    return STD_LOGIC_VECTOR;
```

- Plus some *conversion functions* between similar types (mostly obsolete) and a couple of test functions, notably:
`rising_edge` `falling_edge`

The importance of types

- Take for example a **STD_LOGIC_VECTOR** of size 8 bits

```
my_vector <= "10000001";  
[...]  
if my_vector < 0 then [...]
```

- Is the condition true?
- It depends...
 - If the vector represents an unsigned number, then it is equal to +129 and the condition is false
 - If the vector represents a signed number, then it is equal to -127 and the condition is true
- The operator “<” cannot be defined (has no meaning) for type **STD_LOGIC_VECTOR**
- After a series of “hacks”, the IEEE developed a new package **IEEE.NUMERIC_STD.ALL**

Packages – numeric_std

- *This package contains the definition of the following synthesizable types and functions:*
 - The **signed** and **unsigned** types
 - The logic functions: **and**, **or**, **nand**, **nor**, **not**, **xor**, **xnor**
 - The arithmetic functions: **+**, **-**, *****, **/**, **rem**, **mod**
 - The comparison functions: **<**, **<=**, **>**, **>=**, **=**, **/=**
 - The shift functions: **shift_left**, **shift_right**, **rotate_left**, **rotate_right**
 - Several type conversion functions
- *Note that these functions are only defined for **signed** and **unsigned** types (and not, for example, for **std_logic_vector**)*
- *Once the package has been declared on top of your VHDL file, **these functions can be used without additional declarations***

Complex operators

- One of the key features of VHDL is the possibility of using **complex operators** rather than relying only on logic gates
 - The arithmetic functions: **+**, **-**, *****, **/**, **mod**, **rem**, **abs**
 - The comparison functions: **<**, **<=**, **>**, **>=**, **=**, **/=**
 - The shift functions: **shift**, **rotate**
- Using these operators is fundamental for the design of complex circuits (particularly in datapaths)
- However, arithmetic and comparison operators have different meaning and different hardware implementations depending on the **type** of signal they are applied to

Synthesizable VHDL – NUMERIC.STD

- Using the `IEEE.numeric_std` package and the `signed` and `unsigned` types allows you to use complex operations on data
- IEEE libraries being defined for synthesis, these operators are *fully synthesizable*: they will be transformed into an (optimized) hardware implementation by the synthesis tools. Their use is highly recommended for synthesizable VHDL (much better control of arithmetic operations)
- But be careful! The definition may not necessarily always meet your needs as a designer – for example, *the + operator does not explicitly define a carry out value*

```
function "+" (L, R: UNSIGNED) return UNSIGNED;  
-- Result subtype: UNSIGNED(MAX(L'LENGTH, R'LENGTH)-1 downto 0).  
-- Result: Adds two UNSIGNED vectors that may be of different lengths
```

Synthesizable VHDL – NUMERIC_STD

- *In hardware, **SIGNED** and **UNSIGNED** are exactly equivalent to **STD_LOGIC_VECTOR**, but carry additional information for the synthesis tools, so are not quite identical*
- *In order to use an operator (or a function) on a signal, **the operator must be defined for the specific type of the signal**. For example, from **numeric_std**:*

```
function "+" (L: UNSIGNED, R: UNSIGNED) return UNSIGNED;  
function "+" (L: SIGNED, R: SIGNED) return SIGNED;  
function "+" (L: UNSIGNED; R: NATURAL) return UNSIGNED;  
function "+" (L: NATURAL; R: UNSIGNED) return UNSIGNED;  
function "+" (L: INTEGER; R: SIGNED) return SIGNED;  
function "+" (L: SIGNED; R: INTEGER) return SIGNED;
```

- *As a consequence, their use requires frequent **type conversions** between **std_logic_vector**, **signed/unsigned**, and **integer**. These conversion are realised either through **conversion functions** or **typecasting**.*

Type conversion

- `to_unsigned` / `to_signed`

- These functions convert *integers to unsigned or signed vectors*. Their syntax is:

```
to_unsigned(value, num_bits);
```

```
to_signed(value, num_bits);
```

- For example:

```
signal my_vector : unsigned(7 downto 0);  
[...]  
my_vector <= to_unsigned(34, 8);
```

Type conversion

- `to_integer`

- This function converts *other types to integer*. In synthesizable VHDL, it is used most commonly on signed or unsigned
- Its syntax is:
`to_integer(vector) ;`
- Note that the vector must not have multiple possible integer values. Hence:

```
signal my_vector : std_logic_vector(7 downto 0);  
signal my_integer : integer;  
[...]  
my_integer <= to_integer(my_vector); -- ILLEGAL
```

is illegal, but

```
signal my_vector : unsigned(7 downto 0);  
signal my_integer : integer;  
[...]  
my_integer <= to_integer(my_vector);
```

is correct.

Type casting

- When converting between similar vectors of the same size, *type casting* is used:

```
signal my_vector : std_logic_vector(7 downto 0);  
signal my_unsigned : unsigned(7 downto 0);  
[...]  
my_unsigned <= unsigned(my_vector);  
my_vector <= std_logic_vector(my_unsigned);
```

- For example, to go back to the illegal example of the **to_integer** function, an alternative solution can be:

```
signal my_vector : std_logic_vector(7 downto 0);  
signal my_integer : integer;  
[...]  
my_integer <= to_integer(my_vector); -- ILLEGAL  
my_integer <= to_integer(unsigned(my_vector)); -- OK
```

User-Defined Types

- Pre-defined types are just some of *many possible data types* in VHDL. Others can be *user-defined types*
- Syntax:

```
type type_name is [type definition];  
signal signal_name: type_name;
```

- For synthesis, all types should have an obvious mapping to a binary representation
 - User-defined types:

```
type day_of_the_month is range 0 to 31;  
signal current_day: day_of_the_month;
```

- Synthesis will convert this to binary 0 to binary 31 (5 bit bus)

- User-defined *enumeration* types:

```
type my_door is (open, closed, locked);  
signal current_state: my_door;
```

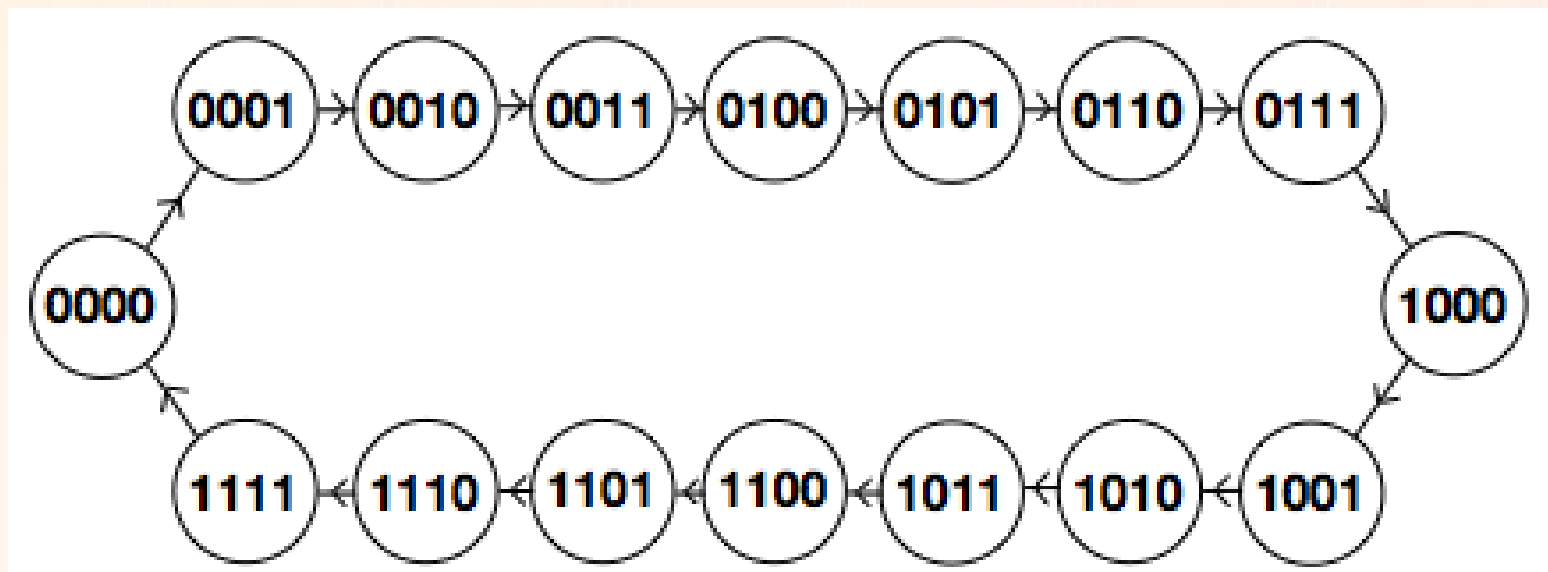
Keep in
mind!

- Synthesis will convert this to binary 0 to binary 2 (2 bit bus)
- Any operator applied to user-defined types must be defined whenever it involves different types

```
my_door <= my_door + 1; -- UNDEFINED OPERATION
```

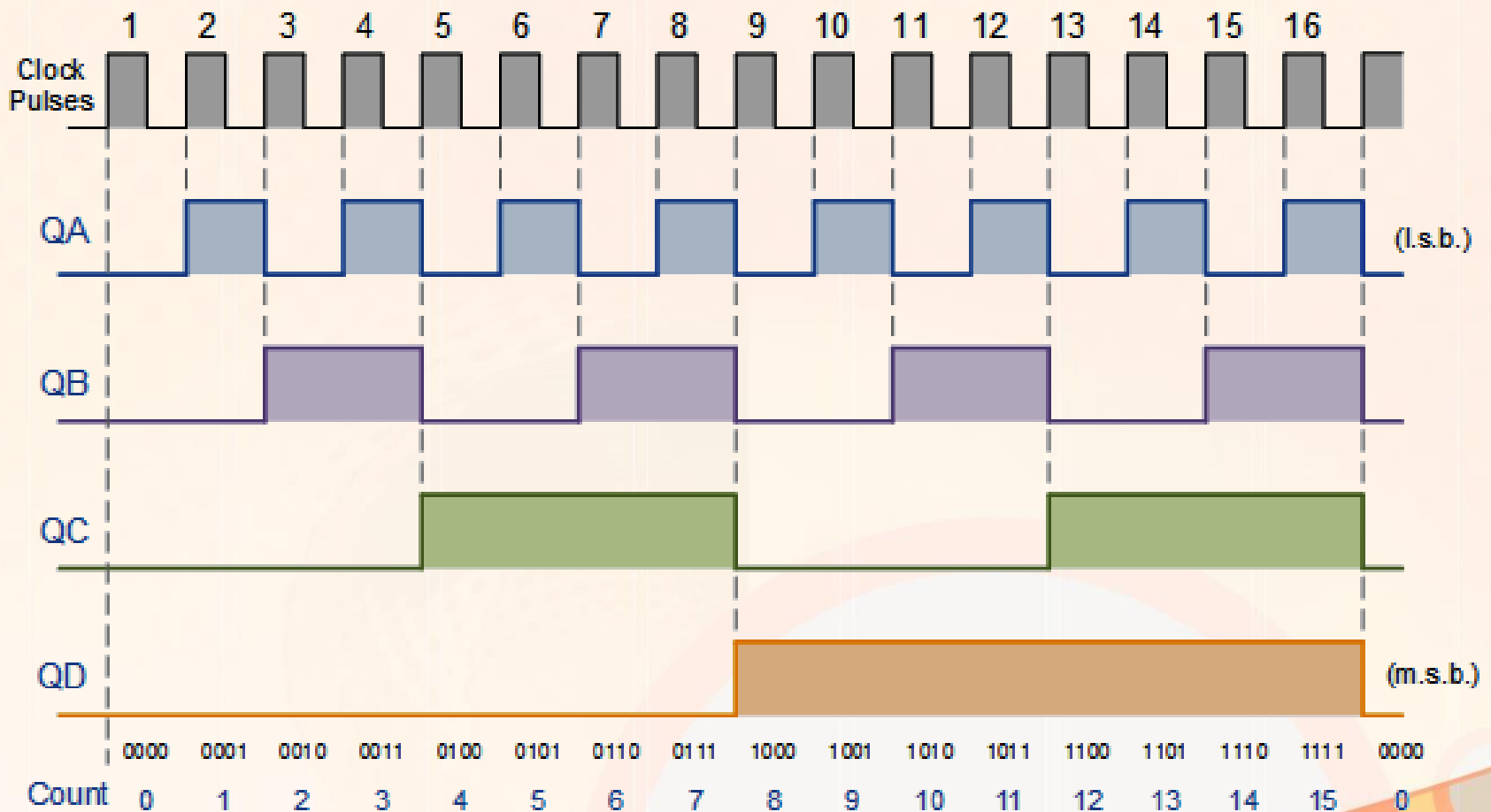
Counters

- Counter: A Sequential Circuit that counts pulses. Used for Event Counting, Frequency Division, Timing, and Control Operations.*



Counters

- Timing Diagram*



Counters

- After (D-type) flip-flops and registers, arguably the most important sequential component for digital designs is the **counter**
- Note that counters are sometimes implemented using type **integer**: this is actually not bad design practice in itself, but requires moderately advanced VHDL skills
 - Type **integer** is often interpreted as a 32-bit bus by synthesis tools, creating a 32-bit counter (rarely needed) – there are strong restrictions on the use of this type for synthesizable VHDL
 - In this module, you should never use type **integer** for counters, but always type **unsigned**



Why do we need unsigned?

Counters – 8-bit unlimited

```
entity CNT_8BIT is
  port (CLK, EN, RST : in STD_LOGIC;
        Q : out UNSIGNED(7 downto 0));
end CNT_8BIT;

architecture arch of CNT_8BIT is
  signal Qint : UNSIGNED(7 downto 0);
begin
  CNT8: process (CLK)
  begin
    if (rising_edge(CLK)) then
      if (RST = '1') then
        Qint <= (others => '0');
      else
        if (EN = '1') then
          Qint <= (Qint + 1);
        end if;
      end if;
    end if;
  end process CNT8;
  Q <= Qint;
end arch;
```

Note the arithmetic operator '+' ! Type UNSIGNED and NUMERIC_STD package required

What happens when the counter reaches 255?

Counters

- Often, counters are *limited* (i.e., they do not use the full range of values that can be represented by their bit size). There are several “proper” implementations of limited counter, and several bad ones.

Counters – 4-bit decimal counter 0 to 9 (1)

```
entity CNT_10 is
  port (CLK, EN, RST : in STD_LOGIC;
        Q : out UNSIGNED(3 downto 0));
end CNT_10;
architecture arch of CNT_10 is
  signal Qint : UNSIGNED(3 downto 0);
begin
  CNT_DEC: process (CLK)
  begin
    if (rising_edge(CLK)) then
      if (RST = '1') then
        Qint <= (others => '0');
      else
        if (EN = '1') then
          if (Qint = 9) then -- or (Qint = "1001")
            Qint <= (others => '0');
          else
            Qint <= (Qint + 1);
          end if;
        end if;
      end if;
    end if;
  end process CNT_DEC;
  Q <= Qint;
end arch;
```

BEST

Counters – 4-bit decimal counter 0 to 9 (1)

```
entity CNT_10 is
  port (CLK, EN, RST : in STD_LOGIC;
        Q : out UNSIGNED(3 downto 0));
end CNT_10;

architecture arch of CNT_10 is
  signal Qint : UNSIGNED(3 downto 0);
begin
  CNT_DEC : process (CLK)
  begin
    if (rising_edge(CLK)) then
      if (RST = '1' or ((Qint = 9) and (EN = '1'))) then
        Qint <= (others => '0');
      else
        if (EN = '1') then
          Qint <= (Qint + 1);
        end if;
      end if;
    end if;
  end process CNT_DEC;
  Q <= Qint;
end arch;
```

FINE

Counters – 4-bit decimal counter 0 to 9 (1)

```
entity CNT_10 is
  port (CLK, EN, RST : in STD_LOGIC;
        Q : out UNSIGNED(3 downto 0));
end CNT_10;

architecture arch of CNT_10 is
  signal Qint : UNSIGNED(3 downto 0);
begin
  CNT_DEC : process (CLK)
  begin
    if (rising_edge(CLK)) then
      if (RST = '1' or Qint = 10) then
        Qint <= (others => '0');
      else
        if (EN = '1') then
          Qint <= (Qint + 1);
        end if;
      end if;
    end if;
  end process CNT_DEC;
  Q <= Qint;
end arch;
```

WRONG

Counters – 4-bit decimal counter 0 to 9 (1)

```
entity CNT_10 is
  port (CLK, EN, RST : in STD_LOGIC;
        Q : out UNSIGNED(3 downto 0));
end CNT_10;
```

```
architecture arch of CNT_10 is
  signal Qint : UNSIGNED(3 downto 0);
begin
  CNT_DEC : process (CLK)
  begin
    if (Qint = 10) then
      Qint <= "0000";
    elsif (rising_edge(CLK)) then
      if (RST = '1') then
        Qint <= (others => '0');
      else
        if (EN = '1') then
          Qint <= (Qint + 1);
        end if;
      end if;
    end if;
  end process CNT_DEC;
  Q <= Qint;
end arch;
```

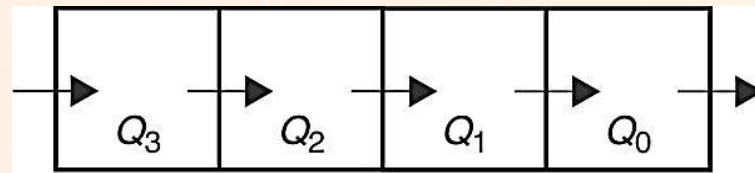
WRONG

Shift Registers

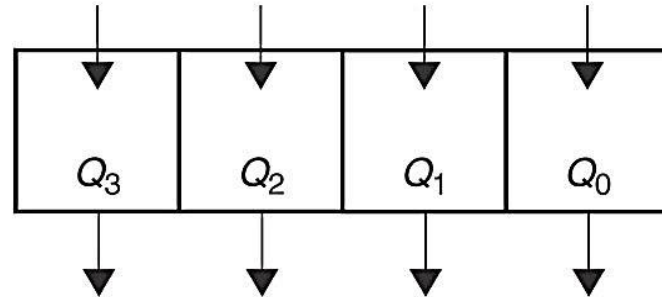
- **Right Shift:** *A movement of data from left to right in the shift register (toward the LSB). One bit shift per clock pulse.*
- **Left Shift:** *A movement of data from right to left in the shift register (toward the MSB). One bit shift per clock pulse.*
- **Rotation:** *Serial shifting (right or left) with the output of the last FF connected to the input of the first. Results in continuous circulation of SR data.*



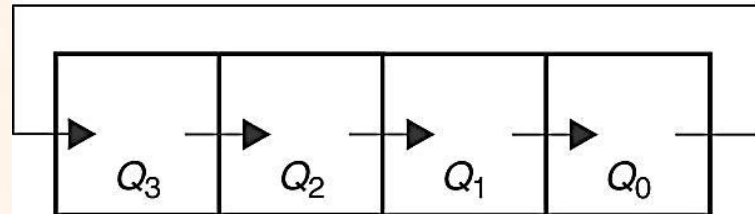
Shift Registers



a. Serial shifting

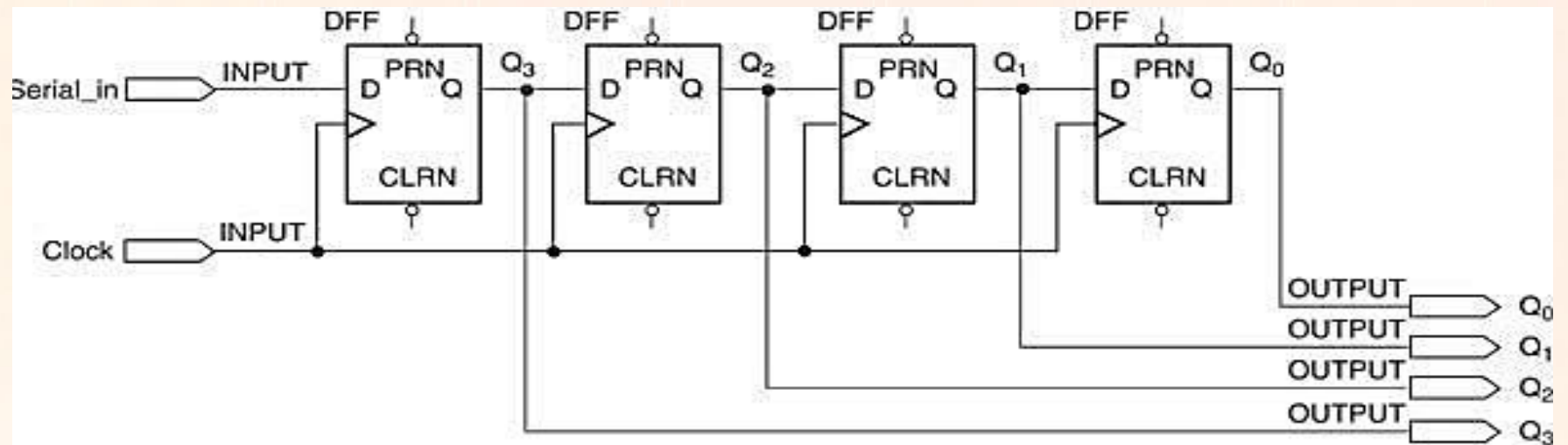


b. Parallel transfer



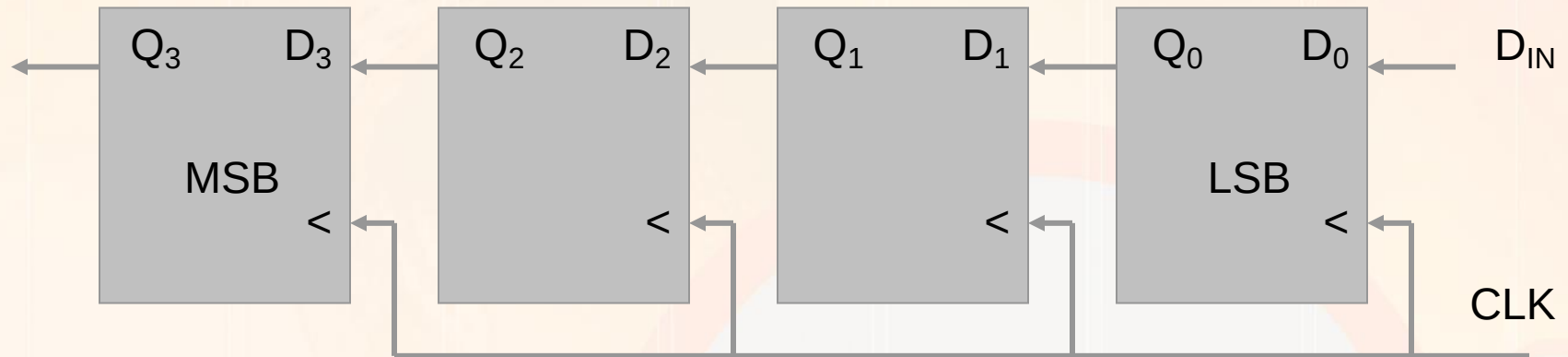
c. Rotation

Shift Registers



Serial Shift Register (SR)

- *A 4-Bit Left Shift Register.*
- *D_{IN} is shifted into the LSB FF and shifted toward the MSB.*



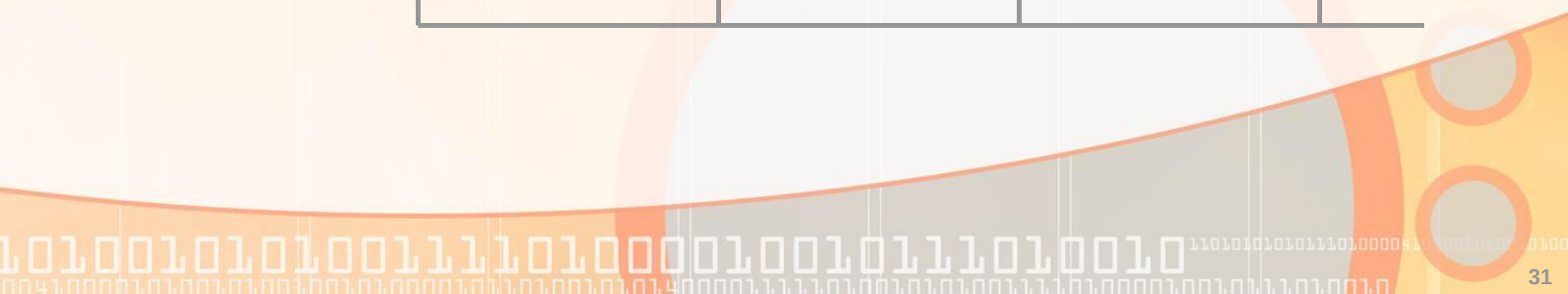
SS Left Shift – 1

The diagram illustrates a 4-bit serial shift register configured for a left shift operation. It consists of four D flip-flops arranged in a chain. The output of each flip-flop is connected to the input of the next flip-flop to its left. The rightmost flip-flop is labeled 'LSB' (Least Significant Bit) and has an input D_{IN} . The leftmost flip-flop is labeled 'MSB' (Most Significant Bit) and has an output line. Each flip-flop has a clock input labeled 'CLK'. The flip-flops are labeled Q_3, D_3 , Q_2, D_2 , Q_1, D_1 , and Q_0, D_0 from left to right. The output of the MSB flip-flop is Q_3 , and the output of the LSB flip-flop is Q_0 . The inputs to the flip-flops are D_3, D_2, D_1, D_0 . The clock signal 'CLK' is connected to the clock input of all flip-flops.

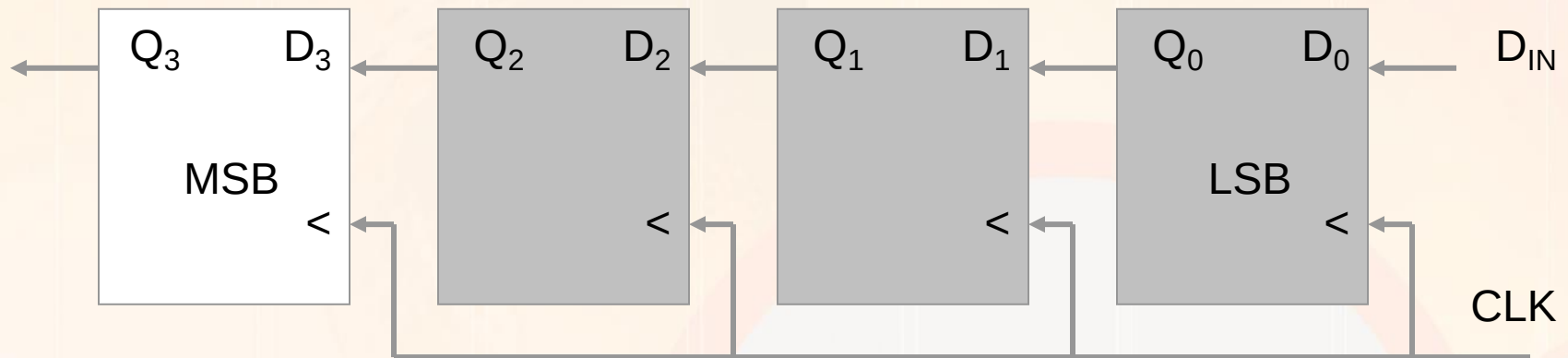
SS Left Shift - 2

The diagram illustrates a 4-bit serial shift register configured for a left shift by 2 positions. It consists of four D flip-flops labeled Q₃ D₃, Q₂ D₂, Q₁ D₁, and Q₀ D₀ from left to right. The output of Q₃ is the MSB (Most Significant Bit), and the output of Q₀ is the LSB (Least Significant Bit). The input D_{IN} is connected to D₀. The clock signal CLK is connected to the clock input of all flip-flops. The data flow is as follows: D_{IN} → D₀ → Q₀ → D₁ → Q₁ → D₂ → Q₂ → D₃ → Q₃. The output of Q₃ is shifted out of the register.

SS Left Shift – 3



SS Left Shift - 4



Bi-Directional Shift Register (see lecture 4)

- *When $DIR = 0$, the path of $Left_Shift_In$ is selected.*
- *When $DIR = 1$, it selects the $Right\ Shift\ In\ Path$.*

$$Q_3 \Leftarrow Q_2 \Leftarrow Q_1 \Leftarrow Q_0 \Leftarrow$$

$$Q_3 \Rightarrow Q_2 \Rightarrow Q_1 \Rightarrow Q_0 \Rightarrow$$

Arrays

- An **array** in VHDL is a type that represents collection of objects of the same type. Its declaration must include the size and the type of object:

```
type myarray is array (31 downto 0) of integer;  
signal array1: myarray;
```

- Arrays are extremely useful constructs. Some of the standard and IEEE types are arrays (**string**, **bit_vector**, **std logic array**, **signed**, **unsigned**). These types are defined in the respective packages and do not need to be re-declared. For example:

```
signal array1: STD_LOGIC_VECTOR (31 downto 0);
```

is equivalent to

```
type my_vector_type is array (31 downto 0) of STD_LOGIC;  
signal my_vector : my_vector_type;
```

- Note that in hardware only one-dimensional arrays make any sense! (with a couple of exceptions)

Arrays

- *Arrays can be declared in either direction and with any finite bound:*

```
input array1: in STD_LOGIC_VECTOR (31 downto 0);  
signal array2: STD_LOGIC_VECTOR (1 to 3);
```

as long as the bounds make sense:

```
input array1: in STD_LOGIC_VECTOR (0 downto 6); --ILLEGAL  
signal array2: STD_LOGIC_VECTOR (3 to 0); --ILLEGAL
```

*For synthesizable VHDL, the use of **to** is strongly discouraged!*

- *Any “slice” of an array can be assigned, including a single element, as long as the two sides have the same size:*

```
array2(1) <= '0';  
array2 <= array1(2 downto 0);  
array1(15 downto 8) <= "01010101";
```

- *Once a direction has been declared, it cannot be changed:*

```
array1(0 to 7) <= "01010101"; --ILLEGAL
```

Arrays

```
signal array1: STD_LOGIC_VECTOR (4 downto 0);  
signal array2: STD_LOGIC_VECTOR (2 downto 0);  
signal a,b,c: STD_LOGIC;  
signal array3: STD_LOGIC_VECTOR (1 downto 0);
```

- Arrays can be defined with the **concatenation operator &**:

```
array2 <= a & b & c;           -- order is important (downto)  
array1 <= array3 & array2;    -- order is important (downto)
```

- In comparison operators, array operands can be of different lengths, but are left-aligned:

```
array1 <= "10011";  
array2 <= "111";  
if (array2 > array 1) then [...]
```

Returns TRUE for unsigned numbers! **So don't do it!**

Arrays

```
signal array1: STD_LOGIC_VECTOR (4 downto 0);  
signal array2 : STD_LOGIC_VECTOR (1 downto 0);  
signal a: STD_LOGIC;
```

- The **others** clause is particularly useful when dealing with arrays:

```
array1 <= array2 & (others => '0');  -- concatenation
```

and in particular:

```
array1 <= (others => '0');  -- reset
```

```
REG: process (CLK)  
begin  
  if (rising_edge(CLK)) then  
    if (RST = '1') then  
      array1 <= (others => '0');  -- instead of "00000";  
    elsif (WEN = '1') then  
      array1 <= Data;  
    end if;  
  end if;  
end process REG;
```

KEEP IN MIND!

Arrays: memories

*An exception to the one-dimension rule is in the definition of **memories** (both ROMs and RAMs).*

```
entity async_rom16x8 is
    Port ( Address : in  UNSIGNED (3 downto 0);
          DataOut  : out  STD_LOGIC_VECTOR (7 downto 0));
end async_rom16x8;

architecture arch of async_rom16x8 is

    type ROM_Array is array (0 to 15) of std_logic_vector(7 downto 0);
    constant Content: ROM_Array := (
        0 => B"00100100",
        1 => B"00000001",
        2 => B"00010010",
        5 => B"11111111",
        6 => B"00000011",
        others => B"00000000");

begin
    DataOut <= Content(to_integer(Address));
end arch;
```

Size of ROM (16x8)

Contents of addresses 0, 1, 2, 5, 6
(B"XXXX") = binary number

0s for all other addresses

Library function

Asynchronous read (no clock)

Arrays: memories

```
entity sync_rom16x8 is
    Port (CLK, REN : in  STD_LOGIC;
          Address : in  UNSIGNED (3 downto 0);
          DataOut  : out  STD_LOGIC_VECTOR (7 downto 0));
end sync_rom16x8;

architecture arch of sync_rom16x8 is
    type ROM_Array is array (0 to 15) of std_logic_vector(7 downto 0);
    constant Content: ROM_Array := (
        0 => B"00100100",
        1 => B"00000001",
        2 => B"00010010",
        5 => B"11111111",
        6 => B"00000011",
        others => B"00000000");
begin
    process (CLK)
    begin
        if (rising edge (CLK)) then
            if (REN='1') then
                DataOut <= Content(to_integer(Address));
            end if;
        end if;
    end process;
end arch;
```

Size of ROM (16x8)

Contents of addresses 0, 1, 2, 5, 6
(B"XXXX") = binary number

0s for all other addresses

Synchronous read
(rising edge)

No reset in a memory

Arrays: memories

```
entity dual_port_ram16x8 is
    Port ( CLK, WEN : in  STD_LOGIC;
           WAddress : in  UNSIGNED (3 downto 0);
           RAddress : in  UNSIGNED (3 downto 0);
           DataIn  : in  STD_LOGIC_VECTOR (7 downto 0);
           DataOut  : out STD_LOGIC_VECTOR (7 downto 0));
end dual_port_ram16x8;
```

Separate read and write ports

```
architecture arch of dual_port_ram16x8 is

    type ram_type is array (0 to 15) of std_logic_vector(7 downto 0);
    signal my_ram: ram_type := (
        0 => X"F3",
        1 => X"06",
        others => X"00");
begin

    DataOut <= tmp_ram(to_integer(RAddress));

    process (CLK)
    begin
        if (rising_edge(CLK)) then
            if (WEN='1') then
                my_ram(to_integer(WAddress)) <= DataIn;
            end if;
        end if;
    end process;
end arch;
```

Signal, not constant for values (optional, rare, initialization)

Asynchronous read

Synchronous write (rising edge)

No reset in a memory

Thank you!

